CSC 6302 Computer Vision

Practical 7 Capstone Report

Road & transport: vehicle counting and classification; pothole severity hint from dashcam images

Student: Munyeme Sinyani
Computer number: 2025024319
University: University of Zambia
Mobile app: https://github.com/Salaam21/blindspot (branch vehicle-counting)

1. Problem and track choice

I chose the Road & transport track: count and classify vehicles from roadside footage, and give a rough pothole severity hint from dashcam-style photos. I already had a phone project (Blindspot) with an on-device SSD MobileNet TFLite model that can see cars, buses, trucks, motorcycles and bicycles. For this practical I specialised that app for traffic counting, and built a Gradio twin in prac7/ that uses YOLOv8n so the marker can try uploads on a laptop.

2. System overview

Two surfaces share the same problem statement. (1) Android Blindspot branch `vehicle-counting`: live camera, filters COCO detections to vehicle classes, shows and speaks a live count. The TFLite weights already live under `app/src/main/ml/`. (2) Gradio `prac7/app.py`: upload a roadside frame, YOLOv8n detects vehicles, draws boxes, returns per-class counts; a second tab runs an OpenCV dark-blob heuristic for pothole severity (low / medium / high) with an explicit coursework disclaimer. Note: Blindspot is native Android/Kotlin, not Flutter.

3. Method

Vehicle classes kept: bicycle, car, motorcycle, bus, truck. Confidence gate defaults to 0.35 on Gradio. Phone frames are often stored sideways, so the Gradio path tries EXIF orientation and alternate rotations and keeps the orientation with the strongest vehicle count. Blurry frames are rejected. Pothole hint: lower-half ROI, threshold dark pixels, morphology, blob count then severity label. This is not a trained pothole network; the report treats it as a severity hint only.

4. Results

On my demo frame (`images/street.jpg`, orientation-corrected) YOLOv8n reported **1 motorcycle** above the gate. Annotated output and the pothole ROI overlay are below. Pytest: 5 checks for count formatting, blur reject, and pothole path.



Figure 1. Vehicle counting annotation (motorcycle detected).

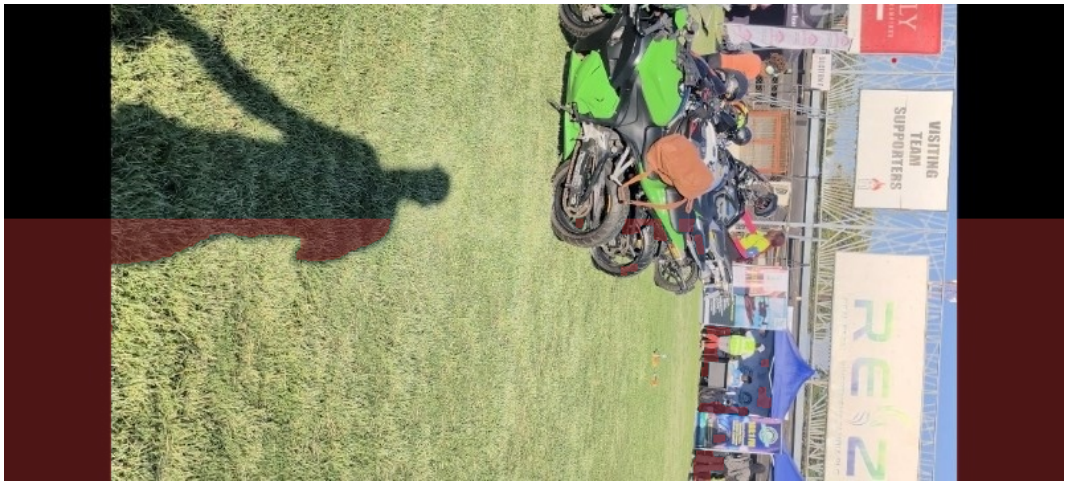


Figure 2. Pothole severity heuristic overlay on lower ROI.

5. Ethics and limits

Counts are model estimates, not official traffic statistics. I do not run number-plate OCR. Night, rain and glare will drop confidence; the UI prefers unsure/blurry over a confident wrong total. Pothole shadows and wet asphalt can look like damage — always verify on site.

6. Repos and how to run

Practicals repo branch: `prac7-road-transport-vehicle-counting`. Mobile: `https://github.com/Salaam21/blindspot` branch `vehicle-counting`. Run: `cd prac7` then `pytest -q`, then `python prac7/app.py`. YOLOv8n downloads on first use (or uses `prac7/weights/`).

7. AI use

Composer helped with Gradio/YOLO glue, Android count UI changes, tests and this report layout. I chose the road-transport track and tied it to my existing Blindspot phone project.